

Scratch Finder 공정 혐의점수 로직 리포트

wafer 단위 good/bad와 정황 y를 이용한 step_seq ranking 모델 - 2026-07-24

본 문서는 wafer별 사용자 판정(good_bad)과 별도 로직으로 계산된 정황 y를 결합하여 공정 단계(step_seq)의 혐의점수를 계산하는 방법을 설명한다. 핵심은 세 가지를 분리하는 것이다. 첫째, Bad wafer가 y=1 정황에 우연 이상으로 몰렸는지 보는 Fisher/hypergeometric evidence, 둘째, Bad miss와 Good false context를 비대칭으로 벌점화하는 quality, 셋째, 실제 관측 wafer 수가 작을 때 점수가 과도하게 커지지 않도록 하는 N_real 보정이다.

1. 입력 데이터와 집계 단위

운영 입력은 하나의 step_seq에 속한 하나 이상의 root_lot_id를 포함할 수 있다. 최소 컬럼은 root_lot_id, wafer_id, good_bad, y이며, step_seq는 컬럼으로 넣거나 함수 인자로 넘긴다. 동일 step_seq의 모든 lot을 하나의 2x2 count table로 합쳐 점수를 한번 계산한다.

```
good_bad_i in {good, bad}
y_i = 1: bad context, y_i = 0: good context
wafer_key_i = (root_lot_id_i, wafer_id_i)
score_group = all rows sharing the same step_seq
```

wafer_id는 lot마다 다시 1부터 시작할 수 있으므로 LOT_A/wafer 1과 LOT_B/wafer 1은 서로 다른 wafer다. 같은 step 안에서 중복을 금지하는 단위는 (root_lot_id, wafer_id) 조합이다. lot 개수 제한은 이 scoring 모듈의 책임이 아니며, 별도의 입력 처리 계층에서 적용한다.

```
n_B1 = count(good_bad = bad, y = 1)
n_B0 = count(good_bad = bad, y = 0)
n_G1 = count(good_bad = good, y = 1)
n_G0 = count(good_bad = good, y = 0)
n_B_real = n_B1 + n_B0
n_G_real = n_G1 + n_G0
N_real = n_B_real + n_G_real
m_real = n_B1 + n_G1
```

N_real은 해당 step에 입력된 모든 lot의 실제 wafer 행을 합한 수다. 뒤에서 추가되는 virtual good은 Fisher evidence 계산에만 쓰며 likelihood quality와 sample-size correction에는 포함하지 않는다.

2. Virtual good prior

Bad-only step처럼 실제 Good wafer가 전혀 없는 경우에도 최소 contrast를 제공하기 위해 모든 process에 동일하게 Good & y=0 pseudo wafer h개를 evidence용 effective table에만 추가한다.

```
virtual_good_h = h = 3
n_B1_eff = n_B1, n_B0_eff = n_B0
n_G1_eff = n_G1, n_G0_eff = n_G0 + h
N_eff = n_B1_eff + n_B0_eff + n_G1_eff + n_G0_eff
B_eff = n_B1_eff + n_B0_eff
m_eff = n_B1_eff + n_G1_eff, x_obs = n_B1_eff
```

prior 크기는 모든 step에 동일하다. 실제 Good contrast가 없는 상황에서도 evidence를 계산할 수 있지만, N_real 보정은 실제 행만 사용하므로 작은 입력이 과도하게 높아지는 현상은 제한된다.

3. Fisher / hypergeometric evidence 유도

귀무가설은 'm_eff개의 y=1 정황이 N_eff개 wafer 위치 중 무작위로 선택되었다'이다. 즉 Bad/Good label은 고정되어 있고, y=1 표시만 무작위로 흩뿌려졌다고 본다. 이때 y=1 위치 안에 들어간 Bad 개수 X는 hypergeometric 분포를 따른다.

```
X ~ Hypergeometric(N_eff, B_eff, m_eff)

P(X = x) = C(B_eff, x) * C(N_eff - B_eff, m_eff - x) / C(N_eff, m_eff)
```

우리가 관심 있는 것은 관측값 x_obs만큼 또는 그보다 더 많이 Bad가 y=1 안에 몰릴 확률이다. 따라서 one-sided upper tail을 evidence의 원천으로 사용한다.

```
p_tail_eff = P(X >= x_obs)
= sum_{x=x_obs}^{min(B_eff, m_eff)} P(X = x)

evidence_eff = -log(max(p_tail_eff, eps))
```

p_tail_eff가 작을수록 무작위로는 설명하기 어렵다는 뜻이다. -log 변환을 쓰면 p-value가 작아질수록 evidence가 선형에 가깝게 커지고, eps로 수치 underflow를 막는다.

Bad-only perfect 예시

```
n_B1=5, n_B0=0, n_G1=0, n_G0=0, h=3
effective table: Bad y=1 = 5, Good y=0 = 3
N_eff=8, B_eff=5, m_eff=5, x_obs=5

p_tail_eff = C(5,5) * C(3,0) / C(8,5)
= 1 / 56
evidence_eff = -log(1/56) = 4.025...
```

single bad wafer는 N_eff=4, B_eff=1, m_eff=1 이므로 p_tail=1/C(4,1)=1/4, evidence 약 1.386이다. 여기에 r_real=1/(1+5)이 곱해져 최종 score는 낮아진다.

4. 비대칭 likelihood와 quality

정황 y가 실제 good/bad와 잘 맞는지의 품질은 별도의 비대칭 likelihood 관점에서 측정한다. 기본값은 Bad wafer가 y=1에 놓일 확률 p_B=0.95, Good wafer가 y=1에 놓일 확률 p_G=0.20 이다. 이 값은 score의 ranking evidence를 직접 대체하지 않고, 잘못 놓인 wafer 비율에 대한 quality penalty를 만든다.

```
p_B = 0.95, p_G = 0.20

log_likelihood = n_B1 * log(p_B)
+ n_B0 * log(1 - p_B)
+ n_G1 * log(p_G)
+ n_G0 * log(1 - p_G)

mean_likelihood = exp(log_likelihood / N_real)
```

mean_likelihood는 진단용이다. 작은 N에서 우연히 완벽히 맞은 경우가 과도하게 높게 보일 수 있으므로, 최종 ranking score는 mean_likelihood를 단독으로 사용하지 않는다.

```
w_B0 = log(p_B / (1 - p_B))
w_G1 = log((1 - p_G) / p_G)

bad_miss_rate = n_B0 / n_B_real
good_false_context_rate = n_G1 / n_G_real

J = 0.7 * w_B0 * bad_miss_rate
+ 0.4 * w_G1 * good_false_context_rate

quality_real = exp(-J)
```

Bad miss는 강하게 본다. p_B=0.95이면 w_B0=log(19)로 크다. Good이 y=1에 놓이는 것은 약한 penalty지만, Good wafer가 많이 y=1에 몰리면 good_false_context_rate가 커져 quality가 낮아진다. n_G_real=0인 Bad-only case에서는 Good penalty 항을 계산하지 않고 Bad miss 항만 사용한다.

5. 최종 score

최종 점수는 evidence, quality, N_real correction의 곱이다. evidence는 '우연 이상으로 Bad가 y=1에 몰렸는지'를 보고, quality는 '그 패턴이 공정 혐의 관점에서 깨끗한지'를 보고, r_real은 '관측 수가 너무 작은지'를 본다.

```
k_N = 5.0
r_real = N_real / (N_real + k_N)

if n_B_real == 0:
    score = 0
else:
    score = r_real * quality_real * evidence_eff
```

n_B_real=0이면 실제 Bad wafer가 없으므로 공정 혐의 score를 만들지 않는다. m_eff=0 또는 contrast가 없는 경우 p_tail_eff는 1로 처리되어 evidence가 0이 된다. virtual good prior가 들어간 뒤에는 Bad-only perfect처럼 원래 no-good contrast였던 케이스도 evidence를 가질 수 있지만, 그 크기는 N_real correction이 함께 제어한다.

참고용 exact count probability

```
log_count_prob_alt = log C(n_B_real, n_B1)
+ n_B1 log(p_B) + n_B0 log(1 - p_B)
+ log C(n_G_real, n_G1)
+ n_G1 log(p_G) + n_G0 log(1 - p_G)

score_exact_alt = r_real * quality_real * (-log(max(count_prob_alt, eps)))
```

이 값은 identical particle 관점에서 count-level probability에 조합항을 포함한 참고 지표다. 기본 ranking은 Fisher/hypergeometric evidence 기반 score를 사용한다.

6. Polars 전용 입출력 코드 양식

DataFrame API는 Polars로 통일되어 있다. `score_single_step_lot_dataframe`은 `pl.DataFrame` 또는 `pl.LazyFrame`을 입력받고, 모든 root lot을 한 step의 count로 합산한 뒤 1행짜리 `pl.DataFrame`을 반환한다. pandas 객체나 dict 출력 선택지는 사용하지 않는다.

```
import polars as pl
from wafer_process_suspicion_sim import (
    SINGLE_STEP_LOT_OUTPUT_COLUMNS,
    score_single_step_lot_dataframe,
)

user_df = pl.DataFrame({
    "root_lot_id": ["LOT_A"] * 5 + ["LOT_B"] * 5,
    "wafer_id": [1, 2, 3, 4, 5, 1, 2, 3, 4, 5],
    "good_bad": [
        "bad", "bad", "good", "good", "good",
        "bad", "good", "good", "good", "good",
    ],
    "y": [1, 1, 0, 0, 0, 1, 0, 0, 0, 0],
})

result_df = score_single_step_lot_dataframe(
    user_df, step_seq="ETCH_010"
)
print(result_df.select(SINGLE_STEP_LOT_OUTPUT_COLUMNS))
```

위 결과는 `pl.DataFrame`이며 `root_lot_count=2`, `root_lot_ids={LOT_A, LOT_B}`, `input_rows=N_real=10`이다. LOT_A/wafer 1과 LOT_B/wafer 1은 서로 다른 wafer로 처리된다.

LazyFrame 입력

```
lazy_result_df = score_single_step_lot_dataframe(
    user_df.lazy(), step_seq="ETCH_010"
)
assert isinstance(lazy_result_df, pl.DataFrame)
```

LazyFrame은 함수 진입 시 `collect()`로 materialize한 뒤 같은 Polars validation과 scoring 경로를 사용한다. 반환 타입은 입력이 eager인지 lazy인지와 관계없이 `pl.DataFrame`으로 고정된다.

여러 step을 한 번에 평가

```
from wafer_process_suspicion_sim import score_process_dataframe

result_df = score_process_dataframe(all_steps_df)
ranked = result_df.select(
    "step_seq", "root_lot_count", "root_lot_ids", "score"
)
print(ranked)
```

`score_process_dataframe`도 `pl.DataFrame` 또는 `pl.LazyFrame`만 받는다. Polars `group_by(step_seq, maintain_order=True)`로 count를 계산하고, 동일 step의 모든 lot을 한 행으로 점수화한 뒤 score 내림차순의 `pl.DataFrame`을 반환한다.

검증과 출력 스키마

중복 wafer identity는 (`step_seq`, `root_lot_id`, `wafer_id`) 기준으로 Polars `group_by`를 사용해 검사한다. `wafer_id` 1..25, `good_bad`의 good/bad 값, `y`의 0/1 값도 Polars expression으로 검증한다. `root_lot_ids`는 `list(str)`, `count`는 `Int64`, `likelihood`와 `score`는 `Float64`로 유지된다. 시뮬레이션 생성 함수와 plot 입력 역시 Polars다.

7. 시뮬레이션 결과 해석

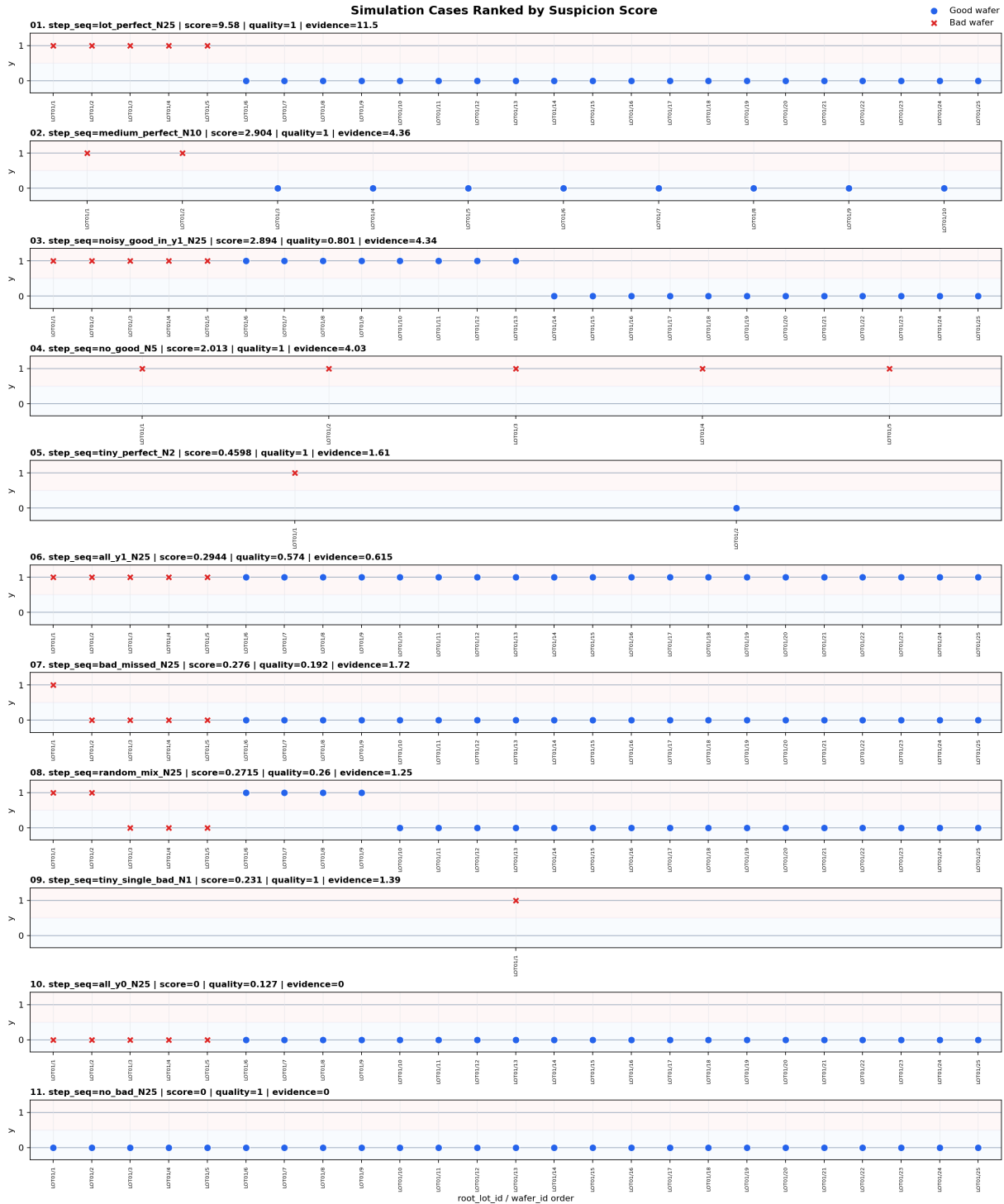
아래 표는 현재 기본 파라미터로 11개 케이스를 score 내림차순 정렬한 결과다. B1/B0/G1/G0는 각각 Bad-y1, Bad-y0, Good-y1, Good-y0 count를 뜻한다.

#	case	N	B	G	B1/B0/G1/G0	quality	evidence	score
1	lot_perfect_N25	25	5	20	5/0/0/20	1	11.5	9.58
2	medium_perfect_N10	10	2	8	2/0/0/8	1	4.36	2.9
3	noisy_good_in_y1_N25	25	5	20	5/0/8/12	0.801	4.34	2.89
4	no_good_N5	5	5	0	5/0/0/0	1	4.03	2.01
5	tiny_perfect_N2	2	1	1	1/0/0/1	1	1.61	0.46
6	all_y1_N25	25	5	20	5/0/20/0	0.574	0.615	0.294
7	bad_missed_N25	25	5	20	1/4/0/20	0.192	1.72	0.276
8	random_mix_N25	25	5	20	2/3/4/16	0.26	1.25	0.271
9	tiny_single_bad_N1	1	1	0	1/0/0/0	1	1.39	0.231
10	all_y0_N25	25	5	20	0/5/0/20	0.127	0	0
11	no_bad_N25	25	0	25	0/0/0/25	1	0	0

조건	의도한 동작	현재 파라미터에서의 해석
Bad-only perfect	virtual good 3개가 contrast를 만들어 score가 0보다 큼	$n_B1=5$ 이면 $p=1/C(8,5)=1/56$, evidence 약 4.03
Bad-only single wafer	evidence는 생기지만 N_real 보정으로 낮게 유지	$n_B1=1$ 이면 $p=1/C(4,1)=1/4$, $r_real=1/(1+5)$
Full lot perfect	실제 Good contrast와 큰 N_real 때문에 가장 높은 축	Good $y=0$ 20개에 virtual good 3개가 더해져 p_tail 이 매우 작음
All real Good also $y=1$	Fisher evidence가 0은 아니어도 quality가 강하게 낮춤	real Good 20개가 $y=1$ 에 있으므로 $good_false_context_rate=1$
No bad	불량 wafer가 없으면 공정 혐의를 만들지 않음	$n_B_real=0$ 이면 $score=0$ 으로 고정

8. Ranked case plot

그림은 score 순위대로 각 케이스를 위에서 아래로 나열한다. x축은 root_lot_id/wafer_id 순서이고, y축은 정황 y=0 또는 y=1이다. 파란 marker는 실제 Good wafer, 빨간 marker는 실제 Bad wafer다.



해석상 가장 중요한 비교는 full lot perfect가 medium/tiny perfect보다 높고, Good이 y=1에 많이 섞인 case는 evidence가 있어도 quality가 낮아져 perfect lot보다 낮아진다는 점이다. Bad miss가 많은 case는 bad_miss_rate penalty 때문에 더 낮아진다.

9. 운영 적용 시 주의점

이 score는 원인 확률 그 자체가 아니라 ranking용 혐의점수다. `p_tail_eff`는 귀무가설 대비 enrichment evidence이고, `quality_real`은 정황 패턴의 공정적 타당성을 벌점화한 계수다. 따라서 score는 `step_seq` 후보를 좁히는 데 사용하고, 최종 root cause 판단은 장비, recipe, 시간대, lot genealogy 등 별도 공정 지식과 함께 보아야 한다.

파라미터를 튜닝할 때는 `h`, `k_N`, `alpha_B`, `alpha_G`를 따로 움직이는 것이 좋다. `h`는 Bad-only contrast의 prior 크기, `k_N`은 작은 랏의 shrinkage 강도, `alpha_B`는 Bad miss penalty, `alpha_G`는 Good false context penalty를 조절한다.

```
Recommended initial parameters
virtual_good_h = 3
k_N = 5.0
p_B = 0.95, p_G = 0.20
alpha_B = 0.7, alpha_G = 0.4
eps = 1e-12
```

구현 파일은 `wafer_process_suspicion_sim.py`이며, notebook 예제는 `wafer_process_suspicion_sim.ipynb`에 있다. PNG 저장은 기본 동작이 아니며, notebook에서는 `plot_ranked_cases(...)`가 figure를 반환하고 `plt.show()`로 inline 표시된다.